

# Raport z realizacji projektu końcowego

Przedmiot: Wprowadzenie do baz danych

Igor Lelito

Jakub Muszalski

Radosław Myśliwy

23 czerwca 2026

## 1 Wstęp i cel projektu

Głównym założeniem projektu było zaprojektowanie i wdrożenie aplikacji w języku Python, która umożliwia sprawną komunikację z bazą danych OpenSky Network. Projekt miał na celu praktyczne wykorzystanie wiedzy z zakresu modelowania systemów bazodanowych, ich integracji z zewnętrznymi interfejsami programistycznymi (API) oraz budowy warstwy analityczno-prezentacyjnej służącej do wizualizacji i interpretacji ruchu lotniczego.

## 2 Architektura systemu

Aplikacja została zbudowana w oparciu o architekturę klient-serwer, z podziałem na warstwę logiczną (backend) oraz interfejs użytkownika (frontend). Do realizacji projektu wykorzystano następujące technologie:

- **Język programowania:** Python - stanowiący fundament zarówno dla logiki strukturalnej programu, jak i interfejsu,
- **Źródło danych:** Baza danych OpenSky Network - dostarczająca rzeczywiste dane o lotach,
- **Framework frontendowy:** Streamlit - umożliwiający szybkie i efektywne tworzenie interaktywnych aplikacji internetowych dla celów analizy danych.

### 2.1 Analiza strukturalna bazy danych

Baza danych została zaprojektowana do przechowywania danych o statkach powietrznych pobieranych z API OpenSky. Dane zostały podzielone na osobne tabele opisujące samoloty, ich pozycje, ruch, kraje, kategorie, obszary monitorowane oraz logi importu.

#### 2.1.1 Opis tabel

Poniżej wymienione są tabele obecne w bazie wraz z ich kolumnami.

**Aircraft** – zawiera informacje o konkretnym samolocie.

- **aircraft\_id** – klucz główny, wewnętrzny identyfikator samolotu w bazie
- **icao24** – identyfikator samolotu z OpenSky, jest unikatowy
- **callsign** – oznaczenie lotu
- **country\_id** – klucz obcy odwołujący się do tabeli „countries”, identyfikator kraju pochodzenia
- **category\_id** – klucz obcy odwołujący się do tabeli „categories”, identyfikator kategorii statku powietrznego

**Categories** – tabela słownikowa mapująca opis i nazwę kategorii do jej numeru id. Kategorie są zaczerpnięte wprost z dokumentacji API.

- **category\_id** – klucz główny, identyfikator kategorii statku powietrznego
- **name, text** – nazwa kategorii
- **description** – opis kategorii

**Aircraft\_position** – zawiera informacje o pozycji samolotu w pewnej chwili czasu.

- **position\_id** – klucz główny, identyfikator rekordu pozycji
- **aircraft\_id** – klucz obcy odwołujący się do tabeli „aircraft”
- **import\_id** – klucz obcy odwołujący się do tabeli „import\_logs”
- **area\_id** – klucz obcy odwołujący się do tabeli „monitored\_areas”, obszar monitorowany, nad którym znajdował się samolot. Jeśli samolot nie znajdował się nad żadnym ze zdefiniowanych obszarów był mu przypisywany ogólny obszar „Reszta świata”
- **time\_position** – czas pozycji
- **longitude** – długość geograficzna
- **latitude** – szerokość geograficzna
- **geo\_altitude** – wysokość geometryczna w metrach
- **baro\_altitude** – wysokość barometryczna w metrach

**Aircraft\_movement** – zawiera informacje o ruchu samolotu w danej pozycji.

- **position\_id** – klucz główny i klucz obcy odwołujący się do tabeli „aircraft\_position”, identyfikator rekordu pozycji
- **velocity** – prędkość w metrach na sekundę
- **true\_track** – kąt lotu w stopniach zgodnie z ruchem wskazówek zegara od północy

**Countries** – tabela słownikowa mapująca nazwę kraju do jego id w bazie.

- **country\_id** – klucz główny, identyfikator kraju
- **name, text** – nazwa kraju

**Monitored\_areas** – zawiera współrzędne geograficzne predefiniowanych obszarów, które są wykorzystywane do filtrowania. Obecna implementacja zawiera współrzędne obszarów nad większymi polskimi miastami.

- **area\_id** – klucz główny, identyfikator obszaru
- **name** – nazwa obszaru
- **lamin** – dolny zakres szerokości geograficznej
- **lomin** – dolny zakres długości geograficznej
- **lamax** – górny zakres szerokości geograficznej
- **lomax** – górny zakres długości geograficznej

**Import\_logs** – tabela techniczna do logowania importu danych z API

- **import\_id** – klucz główny, identyfikator importu
- **imported\_at** – czas importu
- **api\_name** – nazwa API
- **endpoint** – adres API użyty podczas importu
- **records\_received** – liczba rekordów pobranych z API
- **records\_saved** – liczba rekordów zapisanych do bazy danych
- **status** – status (success/error)
- **error\_message** – komunikat błędu, jeśli import się nie udał

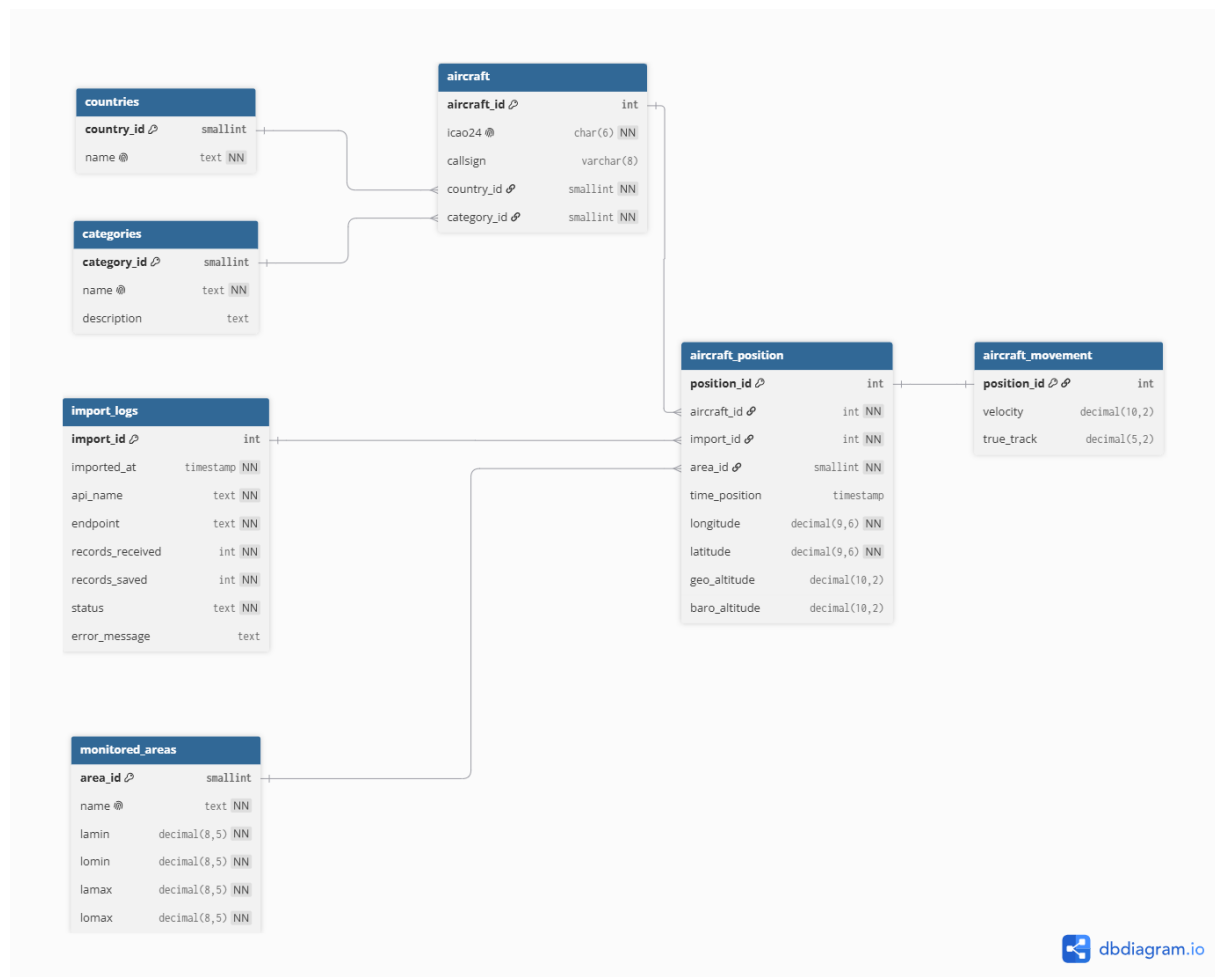
### 2.1.2 Relacje między tabelami

- **countries - aircraft** – relacja jeden-do-wielu. Jeden kraj może mieć wiele przypisanych samolotów, ale jeden samolot ma jeden kraj pochodzenia.
- **categories - aircraft** – relacja jeden-do-wielu. Jedna kategoria może być przypisana do wielu samolotów, ale jeden samolot należy do jednej kategorii.
- **aircraft - aircraft\_position** – relacja jeden-do-wielu. Jeden samolot może mieć wiele zapisanych pozycji w różnych momentach czasu.
- **import\_logs - aircraft\_position** – relacja jeden-do-wielu. Jeden import danych z API może zawierać wiele pozycji samolotów.
- **monitored\_areas - aircraft\_position** – relacja jeden-do-wielu. Jeden obszar monitorowany może obejmować wiele pozycji samolotów.
- **aircraft\_position - aircraft\_movement** – relacja jeden-do-jednego. Jedna pozycja samolotu ma jeden odpowiadający rekord opisujący ruch.

### 2.1.3 Zabezpieczenia danych

- Zastosowanie unikalności dla pola **icao24** zapobiega wielokrotnemu zapisaniu tego samego statku powietrznego.
- Zastosowanie unikalności dla pary **aircraft\_id** oraz **import\_id** w tabeli **aircraft\_position** zapobiega zapisaniu tej samej pozycji samolotu więcej niż raz w ramach jednego importu.
- Zastosowanie unikalności dla pary **aircraft\_id** oraz **time\_position** w tabeli **aircraft\_position** zapobiega zapisaniu tych samych pozycji z API dla jednego samolotu w przypadku gdy w wielu importach znajduje się ta sama pozycja.
- Zastosowanie tabel słownikowych **countries** i **categories** ogranicza powtarzanie tych samych wartości tekstowych oraz zmniejsza ryzyko literówek w danych.

Szczegółowe informacje na temat kolumn jak m.in. ustawienia unikalności, typy danych oraz zabezpieczenia przed wartością Null są zawarte na poniższym schemacie ERD bazy danych oraz w pliku `create_tables.sql` w folderze „database” i w pliku `ERD.dbml` w folderze „docs”. Mając zainstalowane rozszerzenie „dbdiagram” w VSCode lub korzystając ze strony [dbdiagram.io](https://dbdiagram.io) można wygenerować schemat ERD na podstawie kodu w pliku `ERD.dbml`.



Rysunek 1: Schemat ERD bazy danych

Skrypty SQL generujące bazę danych (jeśli nie istnieje) oraz wypełniające tabele „monitored\_areas” oraz „categories” predefiniowanymi rekordami znajdują się w folderze „database” w katalogu projektu.

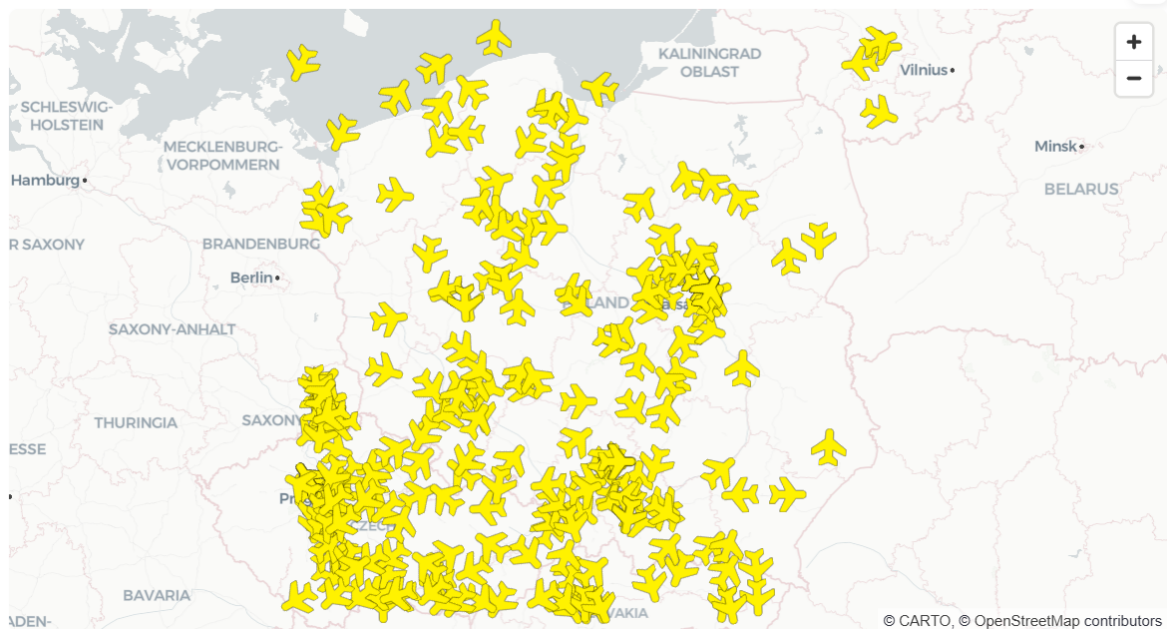
## 2.2 Metody prezentacji danych - dashboard

Dashboard jest aplikacją web’ową składającą się z 3 metod wizualizacji oraz 10 filtrów danych. Można go włączyć z równoległe pracującym importem i zapisem danych do bazy poprzez plik `run.py` w katalogu głównym projektu lub samodzielnie pracując jedynie na już zapisanych danych poprzez wpisanie następującej komendy w terminalu „`python -m streamlit run app/dashboard.py`” znajdując się w katalogu głównym projektu.

### 2.2.1 Wizualizacje

**Mapa samolotów** – pokazuje pozycje samolotów na mapie świata w postaci ikonek samolotu. Orientacja ikony jest zgodna z kątem lotu samolotu. Po najechaniu na ikonę pokazują się informacje o numerze icao24 samolotu oraz kod jego lotu. Ze względu na przejrzystość mapa niezależnie od filtrów zawsze pokazuje tylko najnowszą pozycję danego samolotu. Dodatkowo domyślnie mapa pokazuje tylko pozycje nie starsze niż 15 minut, ale można to zmienić ręcznie ustawiając przedział czasowy za pomocą filtrów. Spowodowane jest to gromadzeniem się samolotów na brzegach monitorowanego obszaru świata (Polska i okolice), ponieważ gdy wylecą za monitorowany obszar, ich najnowsza pozycja jest właśnie na brzegach i pozostaje tam ich martwa, nieaktualizowana wizualizacja.

Mapa samolotów



Rysunek 2: Mapa samolotów

**Karta samolotu** – po kliknięciu na daną ikonkę samolotu, w miejscu mapy pojawią się wykresy ukazujące wartości w czasie jego: prędkości, wysokości geometrycznej oraz

długości i szerokości geograficznej. Do mapy można wrócić klikając przycisk „Powrót do mapy”. Może zająć sytuacja gdy w bazie istnieje tylko jeden zestaw danych na temat samolotu, wtedy wykresy będą miały postać jednego punktu. Wykresy są interaktywne, można je przesuwać (w szczególności w czasie) i skalować.

#### Karta samolotu: 48ad0f ↻



Rysunek 3: Karta samolotu

**Tabela z danymi** – służy do podglądu danych na temat pozycji i ruchu samolotów w bazie. W przeciwieństwie do mapy może wyświetlać również historyczne (nie najnowsze) dane. Jest interaktywna i można w niej m.in. sortować dane rosnąco lub malejąco w kolumnach.

|     | icao24 | oznaczenie lotu | długość geograficzna | szerokość geograficzna | wysokość geometryczna [m] | wysokość barometryczna [m] | prędko |
|-----|--------|-----------------|----------------------|------------------------|---------------------------|----------------------------|--------|
| 223 | 488068 | SPADI           | 18.4997              | 53.2798                | 845.82                    | 723.9                      |        |
| 224 | 48c26c | RYR901          | 20.1946              | 49.3267                | 11475.72                  | 10972.8                    |        |
| 225 | 485873 | KLM1329         | 14.7879              | 51.0679                | 10546.08                  | 10058.4                    |        |
| 226 | 48ada0 | LOT238          | 14.7794              | 53.0417                | 11772.9                   | 11277.6                    |        |
| 227 | 471f8e | WZZ8UW          | 16.4551              | 49.1794                | 9936.48                   | 9387.84                    |        |
| 228 | 4521ba | LOT5PR          | 21.3655              | 49.3074                | 12085.32                  | 11582.4                    |        |
| 229 | 407667 | EZY64CA         | 14.6329              | 49.1331                | 10911.84                  | 10363.2                    |        |
| 230 | 4c01e7 | ASL72D          | 18.4922              | 49.1924                | 11186.16                  | 10652.76                   |        |
| 231 | 4ab564 | SAS765          | 14.1052              | 54.892                 | 4815.84                   | 4572                       |        |
| 232 | 48ad83 | LOT4JH          | 21.2854              | 49.0448                | 12085.32                  | 11582.4                    |        |

Rysunek 4: Tabela z danymi

### 2.2.2 Filtry

Widoczne są w zakładce po lewej stronie dashboardu. Jeśli jest niewidoczna należy ją rozwinąć za pomocą przycisku (dwa grot strzałek) w lewym górnym rogu.

- **„Tylko najnowsze dane samolotu”** – po włączeniu w tabeli pokazują się jedynie najnowsze dane dostępne w bazie.
- **Kraj pochodzenia** – ogranicza dane w wizualizacjach do samolotów pochodzących z danego kraju.
- **Kategoria** – ogranicza dane w wizualizacjach do samolotów tylko z danej kategorii.
- **Lokalizacja** – ogranicza dane w wizualizacjach do tych pochodzących z danego obszaru świata. Obszar można wybrać z predefiniowanych (większe miasta w Polsce) lub zdefiniować go ręcznie zakresem szerokości i długości geograficznych.
- **Minimalna prędkość** – ogranicza dane w wizualizacjach do samolotów z prędkością nie mniejszą niż ustawiona za pomocą suwaka.
- **Wysokość geometryczna** – ogranicza dane w wizualizacjach do samolotów lecących w danym zakresie wysokości geometrycznej. Można ją ograniczyć z obu stron lub z jednej.
- **Wysokość barometryczna** – ogranicza dane w wizualizacjach do samolotów lecących w danym zakresie wysokości barometrycznej. Można ją ograniczyć z obu stron lub z jednej.
- **Icao24** – ogranicza dane w wizualizacjach do tych dotyczących konkretnego samolotu o danym identyfikatorze icao24. Ze względu na działanie mapy oznacza to ukazanie na niej maksymalnie jednego samolotu.
- **Oznaczenie lotu** – ogranicza dane w wizualizacjach do samolotów realizujących dany lot.

- **Przedział czasowy** – ogranicza dane w wizualizacjach do tych pochodzących z danego zakresu czasowego z dokładnością do minuty. Można go ograniczyć z obu stron lub jednej.

Dodatkowo na dole zakładki z filtrami można ustawić wyświetlanie zapytania SQL do bazy danych. Zapytanie oczywiście zmienia się zależnie od użytych filtrów.

### 2.2.3 Użyte zapytania SQL

- Pobranie nazw krajów do wyboru w filtrze „kraj pochodzenia”: `SELECT name FROM countries ORDER BY name`
- Pobranie nazw kategorii do wyboru w filtrze „kategoria”: `SELECT name FROM categories ORDER BY category_id`
- Pobranie nazw monitorowanych obszarów (miast) do wyboru w filtrze „lokalizacja”: `SELECT name FROM monitored_areas ORDER BY name`

Zapytanie do bazy o dane o samolotach jest tworzone dynamicznie zgodnie z wybranymi filtrami. Cała funkcja generująca zapytanie znajduje się w pliku `dashboard_methods.py` w folderze „app”. Gdy filtr „Tylko najnowsze dane samolotu” jest włączony, wspólna część zapytania dla reszty filtrów jest następująca:

```

SELECT
    a.icao24,
    a.callsign AS "oznaczenie lotu",
    ap.longitude AS "długość geograficzna",
    ap.latitude AS "szerokość geograficzna",
    ap.geo_altitude AS "wysokość geometryczna [m]",
    ap.baro_altitude AS "wysokość barometryczna [m]",
    am.velocity AS "prędkość [m/s]",
    c.name AS "kategoria",
    ct.name AS "kraj pochodzenia",
    ap.time_position AS "czas pozycji",
    am.true_track
FROM aircraft AS a
JOIN countries AS ct
    ON a.country_id = ct.country_id
JOIN categories AS c
    ON a.category_id = c.category_id
JOIN aircraft_position AS ap
    ON a.aircraft_id = ap.aircraft_id
JOIN (
    SELECT
        aircraft_id,
        MAX(time_position) AS latest_position
    FROM aircraft_position
    GROUP BY aircraft_id
) AS lp
    ON ap.aircraft_id = lp.aircraft_id AND ap.time_position = lp.latest_position
LEFT JOIN aircraft_movement AS am
    ON ap.position_id = am.position_id

```

Rysunek 5: Fragment zapytania dla włączonego filtra najnowszych danych

Do uzyskania tylko najnowszych danych pozycji każdego samolotu wykorzystano podzapytanie we FROM (JOIN) agregujące dane funkcją MAX. Ta sama część zapytania dla wyłączonego filtra jest następująca:

```

SELECT
    a.icao24,
    a.callsign AS "oznaczenie lotu",
    ap.longitude AS "długość geograficzna",
    ap.latitude AS "szerokość geograficzna",
    ap.geo_altitude AS "wysokość geometryczna [m]",
    ap.baro_altitude AS "wysokość barometryczna [m]",
    am.velocity AS "prędkość [m/s]",
    c.name AS "kategoria",
    ct.name AS "kraj pochodzenia",
    ap.time_position AS "czas pozycji",
    am.true_track
FROM aircraft AS a
JOIN countries AS ct
    ON a.country_id = ct.country_id
JOIN categories AS c
    ON a.category_id = c.category_id
LEFT JOIN aircraft_position AS ap
    ON a.aircraft_id = ap.aircraft_id
LEFT JOIN aircraft_movement AS am
    ON ap.position_id = am.position_id
""

```

Rysunek 6: Fragment zapytania dla wyłączonego filtra najnowszych danych

Dynamiczne dodawanie warunków zgodnie z wybranymi filtrami realizowane jest jak na poniższym przykładzie dla filtra „lokalizacja”:

```

if area:
    if isinstance(area, str):
        area = get_area_cords(area)

    if area[0] is not None:
        conditions.append("ap.latitude >= :min_lat")
        params["min_lat"] = area[0]

    if area[2] is not None:
        conditions.append("ap.latitude <= :max_lat")
        params["max_lat"] = area[2]

    if area[1] is not None:
        conditions.append("ap.longitude >= :min_lon")
        params["min_lon"] = area[1]

    if area[3] is not None:
        conditions.append("ap.longitude <= :max_lon")
        params["max_lon"] = area[3]

```

Rysunek 7: Dynamiczne dodawanie warunków w zapytaniu

Jeśli filtr podany jest w formie krotki z zakresami współrzędnych geograficznych to odpowiednie warunki są dodawane do listy „conditions” a parametry znajdujące się w tych warunkach do słownika „params”. Jeśli filtr ma formę stringa (czyli zostało wybrane konkretne miasto) to współrzędne są pobierane za pośrednictwem funkcji „get\_area\_cords”. Funkcja ta pobiera z bazy informacje o współrzędnych obszaru nad miastem poprzez zapytanie:

```

@st.cache_data
def get_area_cords(area_name):
    query = """--sql
        SELECT lamin, lomin, lamax, lomax
        FROM monitored_areas
        WHERE name = :area_name
    """
    params = {"area_name": area_name}

    df = fetch_query(query, params)

    if df.empty:
        return None

    return df[["lamin", "lomin", "lamax", "lomax"]].iloc[0].tolist()

```

Rysunek 8: Pobieranie informacji o współrzędnych obszaru

Sprawdzenie czy filtry zostały użyte i ich ewentualne uwzględnienie wykonuje poniższy kod:

```

if conditions:
    query += "\nWHERE " + " AND ".join(conditions)

if not params:
    params = None

return fetch_query(query, params), query

```

Rysunek 9: Sprawdzenie i uwzględnienie filtrów

Wrapper „@st.cache\_data” zachowuje w pamięci wynik wywołania funkcji „get\_area\_cords” dla konkretnych parametrów (nazwy obszaru) dzięki czemu zapytanie o dany obszar wysyłane jest tylko raz w ramach całej pracy aplikacji. Takie same wrappery są użyte również w innych funkcjach używanych przez dashboard, niektóre z nich (te wysyłające zapytania) mają ustawiony „czas życia” zapisanego wyniku w pamięci (zmienna „cache\_del\_interval”) o wartości powiązanej z czasem cyklicznego importu danych z API. Dzięki temu rozwiązaniu aplikacja działa sprawniej jednocześnie nie tracąc możliwości ciągłego działania ze zmieniającą się bazą (bez ograniczonego czasu życia funkcja zwracałaby cały czas te same dane dla konkretnych parametrów-filtrów nie „widząc” po-

jawiania się nowych danych). Inne funkcje, np. tworząca połączenie z bazą czy powyższa pobierająca współrzędne mają nielimitowany czas życia.

## 2.3 Architektura backendu i pobieranie danych

Backend aplikacji został zaprojektowany w oparciu o programowanie wielowątkowe. Składa się on z dwóch głównych klas: czytnika API (`ApiReader`) oraz menedżera bazy danych (`DBmanager`). Moduły te komunikują się ze sobą asynchronicznie za pomocą struktury kolejki (`queue`), co zapewnia bezpieczeństwo wątków i zapobiega blokowaniu się procesów.

### 2.3.1 Moduł pobierania danych (`ApiReader`)

Klasa `ApiReader` pełni funkcję producenta danych. Odpowiada za cykliczne odpytywanie zewnętrznego interfejsu OpenSky API i przekazywanie wyników do dalszego przetwarzania.

- **Autoryzacja i inicjalizacja** – podczas tworzenia obiektu, moduł wczytuje poświadczenia (klucz klienta oraz sekret) z pliku konfiguracyjnego `credentials.json`. Pozwala to na uwierzytelniony dostęp do API, co wiąże się z większymi limitami zapytań niż w przypadku korzystania z API bez logowania.
- **Wielowątkowość** – proces pobierania danych zostaje wyodrębniony do osobnego wątku (typu daemon) poprzez uruchomienie pętli `_read_loop`. Pracą wątku steruje obiekt `threading.Event()`, który pozwala na jego bezpieczne zatrzymanie w dowolnym momencie.
- **Ograniczenie obszaru** – zapytania do API są optymalizowane poprzez podanie koordynatów obszaru ograniczającego (bounding box). Moduł pobiera informacje wyłącznie o statkach powietrznych znajdujących się na terenie Polski. Pozwala to na zmniejszenie liczby wysyłanych zapytań, a więc też używanych tokenów API, gdy analizowane są wyłącznie loty nad terytorium kraju.
- **Przekazywanie danych** – po udanym pobraniu wektora stanu, dane trafiają do współdzielonej kolejki, skąd mogą być asynchronicznie odebrane przez system zapisu, a sam wątek usypia na zdefiniowany interwał czasowy.

### 2.3.2 Moduł zapisu i zarządzania bazą (`DBmanager`)

Klasa `DBmanager` jest konsumentem danych oraz głównym interfejsem dostępowym do bazy SQLite. Zarządza zarówno inicjalizacją struktur, zapisem spływających informacji, jak i serwowaniem ich na żądanie.

- **Inicjalizacja i konfiguracja** – przy starcie aplikacja zestawia połączenie z plikiem `data.db`. Moduł automatycznie wykonuje predefiniowane skrypty SQL (`create_tables.sql`, `add_monitored_areas.sql`, `add_obj_categories.sql`), upewniając się, że wszystkie tabele istnieją, a dane słownikowe zostały załadowane. Połączenie ignoruje restrykcje użycia w obrębie jednego wątku (`check_same_thread=False`), co ułatwia zarządzanie aplikacją.

- **Pętla zapisująca** – podobnie jak czytnik API, zapis odbywa się w oddzielnym wątku. Pętla `_save_loop` cyklicznie sprawdza obecność nowych obiektów w kolejce z określonym limitem czasu (`timeout`), który jest dostosowany do interwału pobierania danych.
- **Przetwarzanie i mapowanie logiki** – sercem modułu jest metoda `_save_state_vector`. Wyodrębnia ona z pobranych paczek informacji poszczególne parametry (bez względu na to, czy są słownikami, czy obiektami klas). Pozycje samolotów są w locie porównywane ze współrzędnymi zdefiniowanych obszarów (`monitored_areas`), co pozwala przypisać im odpowiedni identyfikator strefy.
- **Zarządzanie relacjami** – przed zapisaniem samej pozycji moduł weryfikuje istnienie kraju pochodzenia w słowniku, a w razie jego braku dodaje go dynamicznie. Zapis głównego rekordu samolotu wykorzystuje mechanizm aktualizacji w razie konfliktu unikalnego identyfikatora `icao24` (`ON CONFLICT... DO UPDATE`), dbając o zachowanie najświeższego oznaczenia lotu.
- **Bezpieczeństwo transakcyjne** – operacje zapisu obłożone są kontrolą błędów. Naruszenia unikalności wpisów wynikające np. z powielonych odczytów są bezpiecznie ignorowane (`sqlite3.IntegrityError`). W przypadku poważniejszych problemów następuje wycofanie transakcji (`rollback()`), a szczegóły błędu lądują w tabeli logów importu.
- **Integracja z interfejsem graficznym** – moduł udostępnia dedykowaną metodę `fetch_dataframe`, która przyjmuje zapytanie SQL i zwraca wynik bezpośrednio pod postacią ramki danych biblioteki `pandas`. Jest to kluczowy element wykorzystywany do komunikacji backendu z dashboardem prezentacyjnym.

## 2.4 Instrukcja użytkowania i uruchomienia programu

### 2.4.1 Wymagania wstępne i konfiguracja

Aby aplikacja mogła poprawnie funkcjonować i pobierać dane, wymagane jest posiadanie aktywnego konta w serwisie OpenSky oraz lokalne skonfigurowanie poświadczeń. W tym celu należy wykonać następujące kroki:

1. Zarejestrować darmowe konto na stronie OpenSky Network.
2. W zakładce `/my-opensky/account` wygenerować klucz API, pobrać plik `credentials.json` i upewnić się, że jego struktura wygląda następująco:

```
{"clientId": "clientID-example", "clientSecret": "clientSecret-example"}
```

3. Umieścić pobrany plik w głównym katalogu projektu.

### 2.4.2 Uruchomienie aplikacji

Proces uruchamiania został w pełni zautomatyzowany za pomocą skryptu startowego `run.py`. Skrypt ten po wywołaniu automatycznie wykonuje niezbędne czynności przygotowawcze:

- Tworzy nowe wirtualne środowisko w folderze `.venv`.
- Instaluje wszystkie wymagane biblioteki zdefiniowane w projekcie przy użyciu menedżera pakietów `pip`. Lista tych bibliotek znajduje się w pliku `requirements.txt` i może być aktualizowana wraz z rozwojem projektu.

Aby uruchomić aplikację, należy otworzyć terminal w głównym katalogu projektu i wykonać poniższe polecenie:

```
python run.py
```

Program uruchomi wtedy proces backendu systemu i zacznie pobierać dane przy użyciu API. W tym czasie uruchomiona zostanie przeglądarka, a program nawiąże połączenie z lokalną bazą danych i uruchomi interaktywny interfejs użytkownika. W przypadku przerwania działania skryptu, program wstrzymuje zatrzymanie do czasu zakończenia działania procesów potomnych. Zabezpiecza to zapis pobieranych danych i pozwala na w pełni bezpieczne przerwanie działania programu w dowolnym momencie.